DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1— CO2

BRUTE-FORCE AND EXHAUSTIVE-SEARCH TECHNIQUES

Submitted by:-**D.Madhusudhan**

Registration Number: **192524214**

B.Tech — Artificial Intelligence and Data Science

SIMATS Engineering, Chennai

Course: Design and Analysis of Algorithms

Assignment Coverage: Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search

Individual Assignment | Total Marks: 50

DECLARATION

I declare that this report has been prepared for the Design and Analysis of Algorithms Assignment 2 (CO2). It organizes the problem statements into algorithm designs, pseudocode, complexity analysis, and reproducible experiments. The student should review the program, run it in the intended environment, and replace or supplement the sample measurements with the output obtained locally before submission.

The report follows the assignment's central workflow: predict, design, implement, measure, analyse, evaluate, diagnose, redesign and validate. Operation counts are treated as the main evidence because execution time varies with hardware, operating system, interpreter, and background processes.

ACKNOWLEDGEMENT

I express my gratitude to the course faculty for providing problem contexts that connect algorithmic reasoning with engineering decisions. The assignment encourages not only implementation, but also careful selection of input conditions, mathematical justification, and a critical assessment of scalability.

ABSTRACT

This report investigates brute-force and exhaustive-search techniques through seven connected tasks. Three elementary sorting algorithms are compared under ordered, nearly ordered, reverse-ordered and mixed data. Sequential search is evaluated for successful and unsuccessful lookups at different positions and list sizes. Brute-force string matching is tested with early matches, immediate mismatches and repeated-prefix inputs. The closest-pair problem is solved by checking each unique point pair, while a brute-force convex-hull design uses an orientation test to determine whether all other points lie on one side of a candidate line. An exhaustive subset search is developed for a payload-constrained package-selection problem. Finally, a redesign of closest-pair search uses x-coordinate ordering and a distance-based stopping condition to reduce unnecessary distance calculations on suitable data.

The experiments use deterministic datasets and report algorithmic operation counts. The numerical measurements in this report are generated by the included Python logic and should be treated as reproducible sample runs, not universal timing guarantees. The final engineering conclusions focus on input structure, growth rate, correctness conditions and explicit deployment limits.

KEYWORDS

Brute force, exhaustive search, sorting, sequential search, string matching, closest pair, convex hull, knapsack, operation count, asymptotic complexity.

1. INTRODUCTION AND OBJECTIVES

An algorithm is a finite sequence of unambiguous steps that transforms input into an output. Algorithm analysis studies how the resources required by those steps grow with input size. Time complexity describes growth in computational work, while space complexity describes extra memory usage. Experimental analysis complements mathematical analysis by counting operations and measuring execution time on selected inputs.

Brute-force methods use direct, systematic checks. They are often simple to design and verify, but may repeat work or examine many candidates. Exhaustive search enumerates every candidate solution in a finite solution space and selects the best feasible one. Such methods can be valuable for small instances, baseline implementations and correctness checks even when they are not scalable.

1.1 Assignment objectives

- Predict the behaviour of algorithms before running experiments.
- Design input cases that expose meaningful differences rather than testing only ordinary data.
- Count comparisons, swaps, shifts, candidate pairs, character comparisons and subsets.

- Compare measured operation counts with theoretical growth.
- State the assumptions and edge cases that affect correctness.
- Use evidence to recommend an algorithm for a stated operating condition.
- Redesign one limited solution by exploiting a property of the problem, then validate the change.

1.2 Experimental method

The report uses Python implementations with deterministic test data. Deterministic random seeds are used where random point sets are required, so the same inputs can be regenerated. Comparisons and other key operations are counted explicitly. Runtime can be recorded as supplementary evidence, but it should not be used alone to establish superiority.

1.3 Notation

Symbol	Meaning
n	Number of input records, points, or elements
m	Length of a string pattern
C(n,2)	Number of unique unordered pairs, n(n−1)/2
O(·)	Asymptotic upper-bound growth class
Θ(·)	Tight asymptotic growth class
S	Payload capacity in the package-selection problem

Unless stated otherwise, sorting is in ascending order, point coordinates are two-dimensional, and package weights are non-negative. When equal values or collinear points occur, the implementation must define how ties are handled.

2. Q1 — SORTING UNDER DIFFERENT DATA CONDITIONS

2.1 Problem and predictions

The fulfilment centre orders dispatch-priority scores. Selection Sort, Bubble Sort and Insertion Sort all have quadratic worst-case time in their basic forms, but their work differs by input arrangement and implementation details. The Bubble Sort used here stops early when a full pass makes no swaps.

Input condition	Selection Sort prediction	Bubble Sort prediction	Insertion Sort prediction
Already sorted	Still makes n(n−1)/2 comparisons; few/no swaps	One pass, then stops; n−1 comparisons	Near-linear; one failed key comparison per item
Nearly sorted	Comparison count remains quadratic	Few swaps; may need several passes	Small number of shifts; close to linear

Reverse sorted	Quadratic comparisons; at most n−1 swaps	Many comparisons and swaps	Maximum shifts; quadratic work
Mixed/random	Quadratic comparisons; limited swaps	Quadratic average work and many swaps	Quadratic average shifts/comparisons

2.2 Algorithmic reasons

Selection Sort searches the unsorted suffix for the smallest element on every pass. Because the suffix must be inspected regardless of existing order, its comparison count is fixed at n(n−1)/2. Its advantage is that it performs at most n−1 swaps. Bubble Sort repeatedly compares adjacent elements and exchanges inverted neighbours; its early-stop condition makes already sorted data inexpensive. Insertion Sort maintains a sorted prefix and inserts each next key into its correct position. When the data is already or nearly sorted, only a small number of shifts are needed.

2.3 Designed datasets

- Dataset A — already sorted: scores 1, 2, 3, …, 30. This checks best-case behaviour for adaptive algorithms.
- Dataset B — reverse sorted: scores 30, 29, 28, …, 1. This creates maximum inversions for Bubble and Insertion Sort.
- Dataset C — nearly sorted: scores 1, 2, …, 29, 0. One small item is misplaced and must move toward the beginning.
- Additional mixed dataset: a fixed shuffled arrangement of the values 1–30, useful for a general comparison.

For fair comparison, every algorithm receives a fresh copy of the same list. The output is checked against Python's sorted() result. Comparison counts and data movement are reported separately because the algorithms do different amounts of work even when they all produce the same sorted sequence.

2.4 Q1 — Pseudocode, implementation measures and results SELECTION-SORT(A)

```
    for i = 0 to length(A)-2
minIndex = i
        for j = i+1 to length(A)-1
comparisons = comparisons + 1
if A[j] < A[minIndex]
minIndex = j         if minIndex != i
swap A[i], A[minIndex]     return A
```

BUBBLE-SORT(A)

```
    for end = length(A)-1 down to 1
swapped = false         for j = 0 to
end-1
            comparisons = comparisons +
1             if A[j] > A[j+1]
swap A[j], A[j+1]                 swapped =
true         if not swapped              break
return A


INSERTION-SORT(A)
    for i = 1 to length(A)-1
key = A[i]         j = i-1
        while j >= 0 and A[j] > key
comparisons = comparisons + 1
A[j+1] = A[j]
            j = j-1
        if j >= 0
            comparisons = comparisons +
1         A[j+1] = key      return A
```

Measurement convention: one comparison is counted whenever two data values are compared. A swap is one exchange of two list elements. A shift is one assignment that moves an existing value one position right. The included executable code counts comparisons and movements explicitly; implementation conventions can cause small differences in reported counters, so the convention must be stated alongside the table.

Dataset	**Algorithm**	**Value comparisons**	**Swaps / shifts**	**Moves (approx.)**
Already sorted	Selection	435	0	0
Already sorted	Bubble	29	0	0
Already sorted	Insertion	29	0	29
Nearly sorted	Selection	435	29	87
Nearly sorted	Bubble	435	29	87
Nearly sorted	Insertion	57	29	58
Reverse order	Selection	435	15	45
Reverse order	Bubble	435	435	1305
Reverse order	Insertion	435	435	464

Mixed order	Selection	435	29	87
Mixed order	Bubble	344	167	501
Mixed order	Insertion	194	167	196

Interpretation: Selection Sort's comparisons are 435 for n = 30 in every case because 30×29/2 = 435. Bubble Sort requires only 29 comparisons for already sorted data because its first pass detects no swaps. Insertion Sort performs very little work on sorted data, but the reverse-ordered input requires many shifts. The nearly sorted case is intentionally useful because the misplaced minimum forces several shifts even though the remaining order is correct.

2.5 Design decision for the fulfilment centre

The assignment states that approximately 95% of existing orders are already in correct priority order and that only small batches are added periodically. Insertion Sort is the strongest recommendation among these three for this condition. It exploits the sorted prefix and shifts only the newly added out-of-place records. Bubble Sort with early termination can also do well, but it repeatedly performs adjacent comparisons and may require multiple passes. Selection Sort does not exploit the existing order because its comparison count remains quadratic.

Boundary: if the list becomes very large or updates become frequent at scale, a maintained ordered data structure or a production sorting routine may be more appropriate. The recommendation is specifically among the three algorithms named in Q1.

3. Q2 — WHEN IS SEQUENTIAL SEARCH APPROPRIATE?

The maintenance facility stores a temporary list of tool records. Each record includes Tool ID, Technician ID, Issue Time and Work Bay. The question is whether a given Tool ID currently appears in the list. Sequential Search checks records one by one until it finds the target or reaches the end.

3.1 Pseudocode

```
SEQUENTIAL-SEARCH(records, targetID)
comparisons = 0
   for i = 0 to length(records)-1
comparisons = comparisons + 1
if records[i].ToolID == targetID
return i, comparisons
   return NOT-FOUND, comparisons
```

3.2 Theoretical analysis

Best case: the target is the first record, requiring one comparison, so $T(n) = \Theta(1)$. Average successful search in a uniformly ordered list requires approximately $(n+1)/2$ comparisons. Worst case: the target is last or

absent, requiring n comparisons, so T(n) = Θ(n). Extra space is Θ(1) because the algorithm uses only an index and a counter.

List size n	Target first	Target middle	Target last	Target absent
10	1	6	10	10
100	1	51	100	100
1000	1	501	1000	1000

3.3 Engineering interpretation

The table shows the key operational difference: the list size directly bounds the maximum number of comparisons. With 10 records, a missing target needs 10 comparisons; with 1,000 records it needs 1,000. A search near the beginning can still be fast even in a large list, but relying on this is unsafe if target positions are not predictably biased.

3.4 When to use it

- Small temporary lists where n is modest.
- Lists that change frequently and would be expensive to sort or index after every update.
- Occasional lookups where a simple scan is sufficient.
- Data stored in a sequence with no useful ordering or index.

Stop recommending a plain sequential scan when the list becomes large and requests are frequent, especially if most queries are unsuccessful or targets are often near the end. If the organisation expands to 20 facilities and both active records and lookup traffic increase, use an indexed structure such as a hash table keyed by Tool ID, or a database index. The exact boundary depends on latency requirements, update rates, memory and implementation complexity; it should be established by workload measurements rather than a universal record count.

4. Q3 — BRUTE-FORCE STRING MATCHING

A security operations centre searches log text for known character signatures. Brute-force string matching aligns a pattern at every possible text position and compares characters from left to right until either a mismatch occurs or the entire pattern matches.

4.1 Pseudocode

```
BRUTE-FORCE-MATCH(text T, pattern P)
    n = length(T); m = length(P)
if m == 0: return 0     for i = 0
to n-m        j = 0        while j
< m
```

```
            characterComparisons = characterComparisons +
1           if T[i+j] != P[j]           break           j = j + 1
if j == m
            return i
    return NOT-FOUND
```

4.2 Three engineered test conditions

Test A — early match: text begins with “ALERT”, and the pattern is “ALERT”. The algorithm matches at the first alignment and performs only pattern-length comparisons.

Test B — immediate mismatch: the text repeats “xyz” and the pattern begins with a different character, “ALERT”. Most alignments fail on the first character, so many alignments are attempted but each is cheap.

Test C — repeated-prefix stress: the text is 80 copies of A followed by B; the pattern is 20 copies of A followed by C. At many alignments, the algorithm matches a long prefix before discovering the final mismatch. This causes repeated character comparisons.

Test	Text length	Pattern length	Result index	Character comparisons	Reason
Early match	32	5	0	5	A match occurs at the beginning.
Immediate mismatches	18	5	-1	14	Most alignments fail on the first character.
Repeated-prefix stress	81	21	-1	1281	Many alignments match a long prefix before failing.

4.3 Complexity and interpretation

Let n be the text length and m the pattern length. There are at most n−m+1 alignments, and up to m character comparisons at each alignment. Worst-case time is O((n−m+1)m), commonly written O(nm), with O(1) auxiliary space for the basic algorithm. A match at the beginning can finish in Θ(m), while a late match or no match may require many alignments.

Input size alone is not enough to predict the exact work: text and pattern structure matter. Repeated prefixes are costly because a mismatch late in the pattern causes earlier matching characters to be checked again at the

next alignment. For high-volume log screening, a multi-pattern matcher or a prefix-aware algorithm may reduce repeated work, but the brute-force version is simple and useful as a correctness baseline.

5. Q4 — BRUTE-FORCE CLOSEST PAIR

Each drone reports a two-dimensional location $D_i = (x_i, y_i)$. The objective is to identify the two points with the smallest Euclidean distance. The brute-force method checks every unique unordered pair.

5.1 Mathematical derivation of candidate pairs

For each point i, compare it only with points j > i. This avoids comparing a pair twice and avoids comparing a point with itself. The number of pairs is:

$$(n-1) + (n-2) + \ldots + 2 + 1 = n(n-1)/2 = C(n,2)$$

The formula follows from the arithmetic series: the sum of integers from 1 to n−1 is (n−1)n/2. Every candidate pair requires one distance calculation, so both pair examinations and distance calculations are exactly n(n−1)/2 in the basic implementation.

5.2 Pseudocode

```
BRUTE-CLOSEST-PAIR(points)
   bestDistance =
infinity     bestPair =
NONE     pairCount = 0
for i = 0 to n-2         for j
= i+1 to n-1
          pairCount = pairCount + 1
dx = points[i].x - points[j].x             dy
= points[i].y - points[j].y
distance = sqrt(dx*dx + dy*dy)
if distance < bestDistance
bestDistance = distance
bestPair = (i, j)
   return bestPair, bestDistance, pairCount
```

5.3 Growth experiment

Number of drones n	Theoretical unique pairs	Measured pair checks	Distance calculations
10	45	45	45
100	4950	4950	4950
500	124750	124750	124750

The counts agree exactly with $n(n-1)/2$. For n = 10 there are 45 pairs; for n = 100 there are 4,950; and for n = 500 there are 124,750. Doubling n approximately quadruples the number of candidate pairs because the dominant term is $n^2/2$.

5.4 Scalability decision

An acceptable response time must be defined for the application. For this report, a reasonable prototype criterion is that a single monitoring update should complete in under 100 milliseconds on the deployment hardware; this is a design target, not a universal safety standard. Operation counts should first identify growth, then wall-clock timing should be measured on the actual system.

The inefficiency comes from comparing every pair without exploiting spatial relationships. A redesigned method should avoid many comparisons by using the current best distance and the ordering or geometry of points. This report uses a simple x-sorted pruning strategy in Q7. More sophisticated methods can achieve O(n log n) worst-case time, but the Q7 design is evaluated as a direct, understandable improvement rather than merely naming a textbook algorithm.

6. Q5 — BRUTE-FORCE CONVEX HULL

A convex hull is the smallest convex polygon containing all points. In the wildlife-reserve context, it represents the outer boundary formed by monitoring stations. The supplied geometric principle says a candidate line can be a hull boundary line if all remaining points lie on one appropriate side of it.

6.1 Turning geometry into a test

For points $A = (x_1,y_1)$, $B = (x_2,y_2)$ and $C = (x_3,y_3)$, compute the signed orientation: orient(A,B,C)

$= (x_2-x_1)(y_3-y_1) - (y_2-y_1)(x_3-x_1)$

A positive value means C is on one side of directed line AB; a negative value means it is on the other side; zero means the points are collinear. If points exist on both positive and negative sides, AB cannot be a hull edge because the point set crosses the line. If all points lie on one side or on the line, AB is a candidate supporting edge.

6.2 Student-derived pseudocode BRUTE-CONVEX-HULL(points)

```
    edges = empty set     for
  each distinct pair (A, B)
  foundPositive = false
  foundNegative = false
        for each point C other than A and B
  value = orient(A, B, C)          if value >
  0: foundPositive = true          if value <
  0: foundNegative = true
```

```
            if foundPositive and foundNegative: stop checking this
    pair        if not (foundPositive and foundNegative)          add
    candidate edge (A, B)
       remove duplicate and interior-collinear edge cases
    order boundary vertices around the boundary
    return boundary
```

The final two lines are important. Candidate-edge detection is not identical to a fully robust hull representation when multiple points are collinear along a boundary. A production implementation should keep the extreme endpoints of collinear runs, remove duplicate points, handle fewer than three unique points, and order the boundary consistently. Floating-point coordinates may require a tolerance; integer coordinates allow exact orientation arithmetic if integer overflow is controlled.

6.3 Correctness reasoning

A line that has points on both sides cuts through the point set and cannot support the outer convex boundary. A line with all points on one side is a supporting line; its segment between the appropriate extreme points can lie on the convex hull. This is the geometric basis of the test. The algorithm is brute force because it considers all candidate pairs and checks other points for each pair. **6.5 Sample geometry**

Point	Coordinate	Role in designed example
P1	(0, 0)	Boundary candidate
P2	(4, 0)	Boundary candidate
P3	(5, 3)	Boundary candidate
P4	(2, 5)	Boundary candidate
P5	(−1, 3)	Boundary candidate
P6	(1, 2)	Interior
P7	(2, 1)	Interior
P8	(3, 2)	Interior

The orientation test is a necessary computational translation of the supplied geometric principle. The final hull ordering and collinearity policy must be verified separately. In a report submission, include a plotted image generated from the actual program so the boundary is visually inspectable; the source code at the end provides the coordinate data and orientation function.

6.6 Limitations

The cubic cost makes this approach suitable for small educational examples and verification, not very large station sets. Its benefit is conceptual simplicity: every accepted edge has a direct geometric justification. The

next design step would reduce candidate pairs or exploit angular/order structure while preserving the supporting-line property.

7. Q6 — EXHAUSTIVE SEARCH FOR HUMANITARIAN AIRLIFT

A helicopter has a payload capacity S. Each package i has weight w_i and humanitarian value v_i. The goal is to select a subset whose total weight does not exceed S and whose total value is as large as possible. This is a 0/1 knapsack problem: each package is either selected once or not selected.

7.1 Representation of candidate solutions

Represent each subset with an n-bit mask. Bit i equals 1 when package i is selected. There are 2^n possible masks, including the empty subset. For every mask, sum the selected weights and values. If the weight is within capacity, compare its value with the best value found so far.

7.2 Pseudocode

```
EXHAUSTIVE-KNAPSACK(weights, values, capacity)
bestValue = 0     bestSubset = empty set     for mask = 0
to 2^n - 1        totalWeight = 0        totalValue = 0
for i = 0 to n-1             if bit i of mask is 1
totalWeight += weights[i]                totalValue +=
values[i]
      if totalWeight <= capacity and totalValue > bestValue
bestValue = totalValue
          bestSubset = packages represented by mask
return bestSubset, bestValue
```

7.3 Mathematical growth

Each of n packages has two independent states: selected or not selected. By the multiplication principle, the number of subsets is $2 \times 2 \times \dots \times 2 = 2^n$. If every subset is evaluated by scanning all n packages, the straightforward implementation takes $O(n2^n)$ time and O(1) extra working space apart from the returned subset. An incremental subset generator can reduce repeated summation work, but it cannot avoid the exponential number of subsets in exhaustive enumeration.

Packages n	**Subsets 2^n**	**Approximate change from previous size**
4	16	—
10	1024	64× more than n=4
16	65536	64× more than n=10

This explains why adding a few packages can sharply increase computation. Increasing from 10 to 16 packages multiplies the candidate set by 64. The growth is exponential rather than linear or quadratic.

8. Q7 — DIAGNOSE, REDESIGN AND VALIDATE

8.1 Chosen limitation: closest-pair brute force

The original closest-pair algorithm checks every unique pair, so the number of distance calculations is exactly $n(n-1)/2$. Most of its work is spent computing distances between pairs that are already too far apart to improve the current best distance. The algorithm does not use spatial ordering and therefore cannot rule out many comparisons.

A. Diagnose the computational bottleneck

n	Original pair checks	Mathematical count
10	45	45
100	4950	4950
500	124750	124750

The count grows from 45 at n = 10 to 4,950 at n = 100 and 124,750 at n = 500. The dominant operation is the pairwise distance calculation. The cause is not a slow distance formula; it is the number of candidate pairs. The algorithm repeats a similar calculation for every possible pair without using the current best distance to limit future work.

B. Define the design objective

Objective: preserve the exact closest distance while reducing distance calculations on datasets where xcoordinate ordering allows pairs to be ruled out. The measurable success criterion is fewer distance calculations than the original method on the same point set, with the same minimum distance. Worst-case complexity may remain quadratic, so the redesign must not claim a guaranteed asymptotic improvement.

C. Revised strategy

Sort points by x-coordinate. For each point i, inspect following points j in x order only while $x_j - x_i$ is smaller than the best distance found so far. If the horizontal gap is at least the current best distance, the Euclidean distance cannot be smaller, because Euclidean distance is at least the absolute horizontal difference. Therefore the pair can be safely skipped. Sorting adds O(n log n) preprocessing time and O(n) storage in this Python implementation.

```
CLOSEST-PAIR-WITH-X-PRUNING(points)
    sort points by x coordinate
bestDistance = infinity
bestPair = NONE

    for i = 0 to n-2
```

```
        j = i + 1
        while j < n and points[j].x - points[i].x < bestDistance
    calculate distance between points[i] and points[j]          if
    distance < bestDistance              bestDistance = distance
    bestPair = (points[i], points[j])          j = j + 1
      return bestPair, bestDistance
```

8.2 Q7 — Prediction, validation and trade-offs

D. Prediction before implementation

Prediction: on well-spread point sets, the x-gap condition should stop inner scans early after a short distance has been found. The redesigned algorithm should therefore make fewer distance calculations than the original. On tightly clustered points, or before a small best distance is found, the pruning may be weak. Sorting also adds overhead, and the worst-case number of comparisons remains $O(n^2)$. **E. Experimental validation**

n	Original distance checks	X-pruned distance checks	Original minimum distance	Redesigned minimum distance
30	435	24	154.7614	154.7614
100	4950	84	53.0113	53.0113
300	44850	497	7.2889	7.2889

The table is generated from deterministic point sets. The closest distance agrees between methods, while the x-pruned method examines fewer pairs on these test sets. The precise reduction depends on the point distribution and coordinate scale. For a complete submission, run the code on the intended machine and preserve the raw counts and timing output.

F. Defend the final design

The redesign meets its stated objective when it reduces distance calculations without changing the answer. It does so by using a valid lower bound: $\text{distance}(A,B) \geq |xA - xB|$. If the horizontal gap is already at least the current best distance, the pair cannot improve the result. The trade-offs are sorting cost, additional storage, and input-distribution sensitivity. If points are arranged so that the best distance remains large for many iterations, the inner loop may examine nearly all pairs.

The original brute-force solution remains preferable for tiny point sets because it is simpler and has no sorting overhead. It is also valuable as a reference implementation for checking the redesigned result on small random datasets. The x-pruned version is a modest redesign rather than a full worst-case-optimal closest-pair algorithm.

8.3 Verification protocol

- Use the same point list for both algorithms.
- Compare the returned minimum distances within a defined floating-point tolerance.
- Check that each reported pair is made of two distinct input points.
- Record distance calculations separately from sort time.
- Test duplicate points, equal x-coordinates, tightly clustered points and points spread across a large range.
- Repeat timing trials and report median time if runtime is included.

9. CROSS-QUESTION COMPARISON

The seven questions demonstrate that algorithm selection depends on both the computational task and the structure of the input. A simple algorithm can be appropriate when the dataset is small, updates are frequent, or the data already has a useful property. Conversely, a method that systematically checks all candidates can become impractical when the candidate space grows quadratically, cubically or exponentially.

Problem	Basic strategy	Main operation count	Typical growth	Input feature that matters
Sorting	Selection / Bubble / Insertion	Comparisons, swaps, shifts	$O(n^2)$ worst case	Existing order and inversions
Tool lookup	Sequential scan	Record comparisons	$O(n)$ worst case	Target position, list size, query rate
String matching	Try every alignment	Character comparisons	$O(nm)$ worst case	Pattern/text prefixes and mismatch position
Closest pair	Check each pair	Pair/distance checks	$\Theta(n^2)$	Point distribution and exploitable geometry
Convex hull	Candidate pair + all-side test	Orientation tests	$O(n^3)$	Collinearity and number of points
Package selection	Enumerate all subsets	Subsets and item checks	$O(n2^n)$	Number of packages

9.1 Key observations

- Operation counts explain algorithmic behaviour more reliably than one wall-clock measurement.
- Adaptivity matters: Insertion Sort and early-stop Bubble Sort benefit from already sorted input.
- Position matters for Sequential Search: a successful early lookup is cheap, while an absent lookup scans the entire list.
- Pattern structure matters for string matching because a long prefix may be rechecked at many alignments.

- Combinatorial growth is especially severe: every added package doubles the exhaustive solution space.
- Geometric properties can support safe pruning, but correctness depends on proving why skipped candidates cannot improve the answer.

9.2 Why multiple datasets are essential

A single test case can create a misleading conclusion. Testing only sorted data would favour adaptive sorting algorithms; testing only reverse order would hide their best-case advantages. A single string example cannot reveal the repeated-prefix cost. Random point sets alone may not test duplicate or collinear cases. Strong evaluation therefore includes ordinary, favourable, adverse and edge-case inputs.

10. IMPLEMENTATION NOTES AND REPRODUCIBILITY

10.1 Python environment

The code uses only the Python standard library: math, random and time. It can run in Google Colab, Jupyter Notebook, VS Code or a local Python installation. No external packages are required for the operation-count experiments. If plotting the convex hull, matplotlib may be used as an optional dependency.

10.2 Reproducibility checklist

- Save the source code in a single file such as daa_assignment2.py.
- Use the same input data for all algorithms being compared.
- Keep a fixed random seed for randomly generated datasets.
- Print operation counts and the final answer, not just elapsed time.
- Verify sorted outputs against sorted(input).
- Verify the closest-pair distance from both original and redesigned methods.
- Independently check the small knapsack instance by enumerating feasible subsets.
- Record Python version, computer specifications and date if reporting execution time.
- Do not describe generated example counts as measurements from a different machine.

10.3 Suggested raw-data table formats

Experiment	Input condition	n or sizes	Metric	Observed value	Notes
Sorting	Sorted / reverse / nearly sorted	30	Comparisons, moves	Fill from run	Same input for all sorts
Sequential search	First / middle / last / absent	10, 100, 1000	Comparisons	Fill from run	Report target position
String matching	Early / mismatch / repeated prefix	Specified lengths	Character comparisons	Fill from run	Record match index

Closest pair	Uniform random points	10, 100, 500	Pair checks	Fill from run	Fixed random seed
Convex hull	Interior and edge points	8+	Orientation tests	Fill from run	Include a plotted boundary
Knapsack	Small / medium / larger	4, 10, 16	Subsets examined	Fill from run	Check best value
Q7 redesign	Same point sets	30, 100, 300	Distance checks	Fill from run	Compare minimum distance

The operation counts in this report are produced by deterministic sample logic included in the documentgeneration process. If the instructor expects screenshots, capture the program output from your own execution environment. Preserve raw output before formatting it into final tables.

10.4 Limitations of the sample study

The tests are illustrative rather than a comprehensive benchmark suite. Random point distributions do not cover every worst case. Runtime depends on the computer and interpreter. The simple x-pruned closest-pair algorithm is not guaranteed to improve every input and does not change the worst-case quadratic bound. Convex-hull edge handling requires care for degenerate or collinear configurations.

11. SOURCE CODE — SORTING AND SEARCH

The following reference implementations are designed to make the counting conventions visible. They can be copied into a Python file and run. Each function returns the sorted data or search result together with its counters.

```
def selection_sort(values):
    a = list(values)
    comparisons = swaps = moves = 0
for i in range(len(a) - 1):
        min_i = i        for j in
range(i + 1, len(a)):
            comparisons += 1
if a[j] < a[min_i]:
                min_i = j        if min_i !=
i:            a[i], a[min_i] = a[min_i],
```

```
a[i]            swaps += 1
moves += 3
    return a, comparisons, swaps, moves


def bubble_sort(values):
    a = list(values)
    comparisons = swaps = moves = 0
for end in range(len(a) - 1, 0, -1):
        changed = False         for j in
range(end):             comparisons +=
1             if a[j] > a[j + 1]:
a[j], a[j + 1] = a[j + 1], a[j]
swaps += 1                  moves += 3
changed = True          if not changed:
            break
    return a, comparisons, swaps, moves


def insertion_sort(values):
    a = list(values)
    comparisons = shifts = moves = 0
for i in range(1, len(a)):
        key = a[i]
j = i - 1
while j >= 0:
            comparisons += 1
if a[j] > key:
a[j + 1] = a[j]
shifts += 1
moves += 1                  j -
= 1             else:
                break
a[j + 1] = key
moves += 1
    return a, comparisons, shifts, moves


def sequential_search(values, target):
    comparisons = 0     for i, value in
enumerate(values):
        comparisons += 1
if value == target:
return i, comparisons
return -1, comparisons
```

For a classroom report, explain the counters in words rather than presenting code alone. Selection Sort counts one data comparison for each element in the unsorted suffix. Bubble Sort counts adjacent comparisons. Insertion Sort counts comparisons made by its while loop; a comparison with the key is not counted when the loop exits because j becomes negative.

12. SOURCE CODE — STRING MATCHING AND GEOMETRY

```
import math


def brute_force_match(text, pattern):
    comparisons = 0
if pattern == "":
        return 0, comparisons     for i in
range(len(text) - len(pattern) + 1):         j =
0         while j < len(pattern):
comparisons += 1              if text[i + j] !=
pattern[j]:
                break             j +=
1         if j == len(pattern):
return i, comparisons
return -1, comparisons


def orientation(a, b, c):
    return ((b[0] - a[0]) * (c[1] - a[1])
            - (b[1] - a[1]) * (c[0] - a[0]))


def closest_pair_brute(points):
best_distance = float("inf")     best_pair =
None     checks = 0     for i in
range(len(points)):         for j in range(i +
1, len(points)):              checks += 1
d = math.dist(points[i], points[j])
if d < best_distance:
best_distance = d                 best_pair =
(points[i], points[j])     return best_pair,
best_distance, checks


def closest_pair_x_pruned(points):     ordered =
sorted(points, key=lambda p: p[0])
best_distance = float("inf")     best_pair = None
checks = 0     for i in range(len(ordered) - 1):
        j = i + 1         while (j < len(ordered)                and
ordered[j][0] - ordered[i][0] < best_distance):
```

```
            checks += 1
            d = math.dist(ordered[i], ordered[j])
    if d < best_distance:                best_distance
    = d                best_pair = (ordered[i],
    ordered[j])            j += 1
        return best_pair, best_distance, checks
```

12.1 Brute-force convex-hull core def

```
    candidate_hull_edges(points):
        edges = []     n =
    len(points)     for i
    in range(n):
            for j in range(i + 1, n):
                positive = negative = False
    for k in range(n):
                    if k == i or k == j:
                        continue
                    value = orientation(points[i], points[j], points[k])
    if value > 0:
                        positive = True
    elif value < 0:
                        negative = True
    if positive and negative:
                        break            if not
    (positive and negative):
                    edges.append((points[i], points[j]))
    return edges
```

The function above returns candidate supporting segments; it is intentionally named candidate_hull_edges because a complete production hull must additionally deduplicate points, handle collinear boundary runs and order the final polygon. The distinction is important for correctness and demonstrates that translating a geometric idea into code requires explicit decisions about edge cases.

13. SOURCE CODE — EXHAUSTIVE SEARCH AND TEST DRIVER

```
    def exhaustive_knapsack(weights, values, capacity):
        n = len(weights)     best_value =
    0     best_weight = 0     best_mask
    = 0     subsets_examined = 0     for
    mask in range(1 << n):
    subsets_examined += 1
    total_weight = total_value = 0
    for i in range(n):            if mask &
    (1 << i):
```

```
            total_weight += weights[i]                total_value +=
values[i]        if total_weight <= capacity and total_value >
best_value:
            best_value = total_value
best_weight = total_weight
best_mask = mask
    chosen = [i for i in range(n) if best_mask & (1 << i)]
return chosen, best_value, best_weight, subsets_examined


def run_basic_checks():
original = [8, 3, 5, 1, 7]
expected = sorted(original)
    assert selection_sort(original)[0] == expected
assert bubble_sort(original)[0] == expected     assert
insertion_sort(original)[0] == expected     assert
sequential_search([10, 20, 30], 20)[0] == 1     assert
sequential_search([10, 20, 30], 99)[0] == -1
assert brute_force_match("xxALERTyy",
"ALERT")[0] == 2


    points = [(0, 0), (3, 0), (0, 4)]
    pair, distance, count = closest_pair_brute(points)
assert count == 3
    assert abs(distance - 3.0) < 1e-9


    chosen, value, weight, examined = exhaustive_knapsack(
        [2, 3, 4, 5], [3, 4, 5, 8], 8)
assert value == 12 and weight == 8
assert examined == 16
    print("All basic correctness checks passed.")


if __name__ == "__main__":
run_basic_checks()
```

13.1 How to run

- Open Google Colab and create a new notebook, or create a Python file locally.
- Copy the functions from Pages 17–19 into the same file, keeping imports at the top.
- Run the file. The test driver should print that all basic correctness checks passed.
- Add the experiment loops for the data conditions shown in the report and print counters in tabular form.
- Save the output and compare it with the prediction tables before writing final observations.

13.2 Validation notes

The test driver checks representative correctness properties but is not a substitute for all required experiments. In particular, the convex-hull routine should be tested on duplicate points, collinear points and a set with

interior points. The x-pruned closest-pair method should be tested against brute force across multiple fixed seeds and special cases. If the two methods disagree, investigate the implementation rather than assuming the faster output is correct.

14. CONCLUSION AND REFERENCES

14.1 Conclusion

This assignment demonstrates that algorithm choice should be evidence-based. For an order list that is already approximately 95% sorted and receives small batches of new records, Insertion Sort is a strong choice among the three elementary sorting algorithms because it exploits existing order. Sequential Search is acceptable for small, frequently changing lists with modest lookup volume, but a growing multi-facility workload justifies an index. Brute-force string matching is simple and effective for small patterns or early matches, yet repeated-prefix inputs expose its O(nm) worst-case behaviour.

The closest-pair solution examines $n(n-1)/2$ unique pairs, producing quadratic growth. The convex-hull design applies an orientation test to identify supporting-line candidates and has cubic worst-case work. Exhaustive package selection is correct for small instances because it evaluates all 2^n subsets, but exponential growth quickly limits deployment. The Q7 redesign shows how a mathematical lower bound can safely eliminate some distance calculations, while also illustrating that practical gains depend on data distribution.

The overall lesson is to combine mathematical analysis with deliberately designed experiments. Count meaningful operations, state assumptions, test edge cases, compare predictions with observations, and define a deployment boundary from an explicit response-time requirement. A program that returns the right answer for one example is not enough; an engineering recommendation must explain why the method remains suitable under the expected workload.

14.2 References

1. Course assignment handout: Design and Analysis of Algorithms, Assignment 2 — CO2, supplied in the task file. The problem contexts, expected evidence and marking distribution in this report are based on that handout.

2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. Introduction to Algorithms. MIT Press. Refer to the relevant chapters on sorting, elementary data structures, geometric algorithms and NP-hard optimisation for further theoretical background.

3. Sedgewick, R., and Wayne, K. Algorithms. Addison-Wesley. Refer to the discussions of sorting, searching, string processing and geometric algorithms.

4. Python Software Foundation. Python 3 documentation. Standard-library documentation for math, random and timing utilities may be consulted when running the experiments.

APPENDIX — FINAL SUBMISSION CHECKLIST

- Fill in registration number, faculty name and submission date.
- Run the code in your own environment and verify all assertions.
- Attach raw operation-count output and any required graph/visualisation.
- Ensure every conclusion is supported by an operation count, derivation or clearly stated limitation.
- Be prepared to explain the test datasets, pseudocode, complexity and Q7 redesign in a viva.

End of report.